

Modül 23

Build & Deployment

Production-Ready Pipeline

Modül:	23 / 30
Ders Sayısı:	7 ders
Seviye:	İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 23'e Hoş Geldin

Code'unu kullanıcılara nasıl ulaştırırsın? Bu modülde production build optimizations, environment management, Docker, CI/CD pipelines, ve modern deployment platformlarını işleyeceğiz.

Öğrenecekler:

- ✓ Production build optimizations
- ✓ Environment management
- ✓ Docker ile containerization
- ✓ Nginx config
- ✓ GitHub Actions CI/CD
- ✓ Deployment platformları (Vercel, Netlify, Cloud)
- ✓ Monitoring ve error tracking (Sentry)

Production Build

ng build production optimizasyonları.

□ Production Build Komutu

```
# Default production build
ng build

# Bayraklar
ng build --configuration=production
ng build --base-href=/my-app/
ng build --source-map=true          # Debug için source map
ng build --aot=true                 # Ahead-of-Time compilation (default)
ng build --build-optimizer=true     # Tree-shaking + dead code elimination
```

□ Build Output

```
dist/my-app/
  browser/
    index.html
    main-[hash].js          # App code
    polyfills-[hash].js    # Browser polyfills
    runtime-[hash].js      # Webpack runtime
    styles-[hash].css
    assets/                 # Statik dosyalar

# Hash → Cache busting (browser her deploy'da yeni file alır)
```

⚡ Optimizations (Otomatik)

- ✓ AOT compilation — template'leri compile time'da TypeScript'e çevir
- ✓ Tree shaking — kullanılmayan code'u at
- ✓ Minification — boşluk/comment kaldır
- ✓ Mangling — variable isimlerini kısalt
- ✓ Lazy chunks — lazy loaded route'lar ayrı bundle
- ✓ Asset hashing — cache busting

□ Bundle Size Hedefleri

Bundle	İdeal	Maksimum
Main bundle	< 200 KB	< 500 KB

Initial total	< 500 KB	< 1 MB
Lazy chunk	< 100 KB	< 300 KB
Total	< 2 MB	< 5 MB

Environment Management

Dev, staging, production için farklı ayarlar.

□ Environment Dosyaları

```
// src/environments/environment.ts (dev)
export const environment = {
  production: false,
  apiUrl: 'http://localhost:3000/api',
  enableDebug: true,
  features: {
    chat: true,
    analytics: false,
  }
};

// src/environments/environment.staging.ts
export const environment = {
  production: false,
  apiUrl: 'https://staging-api.example.com',
  enableDebug: true,
  features: { chat: true, analytics: true }
};

// src/environments/environment.prod.ts
export const environment = {
  production: true,
  apiUrl: 'https://api.example.com',
  enableDebug: false,
  features: { chat: true, analytics: true }
};
```

⚙️ angular.json — File Replacements

```

{
  "configurations": {
    "production": {
      "fileReplacements": [{
        "replace": "src/environments/environment.ts",
        "with": "src/environments/environment.prod.ts"
      }],
      "optimization": true,
      "outputHashing": "all",
      "sourceMap": false,
      "namedChunks": false,
      "extractLicenses": true,
      "vendorChunk": false,
      "buildOptimizer": true
    },
    "staging": {
      "fileReplacements": [{
        "replace": "src/environments/environment.ts",
        "with": "src/environments/environment.staging.ts"
      }]
    }
  }
}

```

□ Kullanım

```

ng build --configuration=production
ng build --configuration=staging

// Code'da
import { environment } from './environments/environment';

if (environment.features.analytics) {
  // Track
}

```

Docker Containerization

Angular app'ini Docker container'da paketle.

❏ Dockerfile (Multi-Stage)

```
# Stage 1: Build
FROM node:20-alpine AS build

WORKDIR /app
COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# Stage 2: Serve
FROM nginx:alpine

# Custom nginx config
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Build output
COPY --from=build /app/dist/my-app/browser /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

⚙️ nginx.conf

```

server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    # Gzip compression
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_types text/plain text/css text/xml text/javascript
        application/javascript application/json;

    # Cache static assets
    location ~* \.(?:css|js|jpg|jpeg|gif|png|ico|woff|woff2)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # SPA routing – fallback to index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;

    # API proxy (opsiyonel)
    location /api/ {
        proxy_pass http://backend:3000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

```

❏ docker-compose.yml


```
version: '3.8'

services:
  app:
    build: .
    ports:
      - "80:80"
    environment:
      - NODE_ENV=production
    restart: unless-stopped

  backend:
    image: my-backend:latest
    ports:
      - "3000:3000"

# Database, redis, etc.
```

□ Build & Run

```
# Build image
docker build -t my-angular-app .

# Run container
docker run -p 80:80 my-angular-app

# With compose
docker-compose up -d
docker-compose logs -f
```

GitHub Actions CI/CD

Otomatik build, test, deploy pipeline.

⚙️ .github/workflows/ci.yml — Test Job

```
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - run: npm ci
      - run: npm run lint
      - run: npm run test -- --watch=false --browsers=ChromeHeadless
      - run: npm run e2e -- --headless
```

⚙️ Build & Deploy Jobs

```
build:
  needs: test
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: '20'
        cache: 'npm'

    - run: npm ci
    - run: npm run build

    - uses: actions/upload-artifact@v4
      with:
        name: dist
        path: dist/

deploy:
  needs: build
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/download-artifact@v4
      with:
        name: dist

    - name: Deploy to Vercel
      uses: amondnet/vercel-action@v25
      with:
        vercel-token: ${ secrets.VERCEL_TOKEN }
        vercel-args: '--prod'
```

Deployment Platformları

Modern hosting seçenekleri.

Platformlar

Platform	En iyi	Fiyat
Vercel	Next.js benzeri, Angular OK	Free → \$20/mo
Netlify	Statik + serverless	Free → \$19/mo
Cloudflare Pages	Performance, edge	Free → \$20/mo
Firebase Hosting	Google ecosystem	Free → pay-as-go
AWS S3 + CloudFront	Enterprise	Pay-as-go
Azure Static Web Apps	Microsoft ekosistem	Free → pay-as-go

Vercel Deploy

```
# vercel.json
{
  "version": 2,
  "builds": [
    { "src": "package.json", "use": "@vercel/static-build" }
  ],
  "routes": [
    { "handle": "filesystem" },
    { "src": "/*.*", "dest": "/index.html" }
  ]
}

# Deploy
npm install -g vercel
vercel # First time
vercel --prod # Production
```

Netlify Deploy

```
# netlify.toml
[build]
  command = "npm run build"
  publish = "dist/my-app/browser"

[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200

# CLI
npm install -g netlify-cli
netlify deploy
netlify deploy --prod
```

Monitoring & Error Tracking

Production'da neler oluyor görmek.

❑ Sentry Setup

```
npm install @sentry/angular

// main.ts
import * as Sentry from '@sentry/angular';
import { ErrorHandler } from '@angular/core';

Sentry.init({
  dsn: 'https://abc@sentry.io/123',
  environment: environment.production ? 'production' : 'dev',
  integrations: [
    Sentry.browserTracingIntegration(),
    Sentry.replayIntegration(),
  ],
  tracesSampleRate: 0.1, // 10% performance trace
  replaysSessionSampleRate: 0.1, // 10% session replay
  replaysOnErrorSampleRate: 1.0, // 100% error replay
});

// app.config.ts
providers: [
  { provide: ErrorHandler, useValue: Sentry.createErrorHandler() }
]
```

❑ Web Vitals

```
import { onCLS, onFID, onLCP, onFCP, onTTFB } from 'web-vitals';

function sendToAnalytics(metric: any) {
  // Google Analytics, custom backend, etc.
  fetch('/api/analytics', {
    method: 'POST',
    body: JSON.stringify(metric)
  });
}

onCLS(sendToAnalytics); // Cumulative Layout Shift
onFID(sendToAnalytics); // First Input Delay
onLCP(sendToAnalytics); // Largest Contentful Paint
onFCP(sendToAnalytics); // First Contentful Paint
onTTFB(sendToAnalytics); // Time to First Byte
```

Core Web Vitals Hedefleri

Metric	İyi	Kötü
LCP	< 2.5s	> 4.0s
FID	< 100ms	> 300ms
CLS	< 0.1	> 0.25
FCP	< 1.8s	> 3.0s
TTFB	< 0.8s	> 1.8s

Production Checklist

Deploy etmeden önce kontrol edilecekler.

☐ Pre-Deploy Checklist

- ✓ Environment production config ile build
- ✓ Bundle size hedef altında (analiz et)
- ✓ Lighthouse score: Perf 90+, A11y 100, SEO 100
- ✓ Console log'lar production'da temizlendi
- ✓ Source map kararı verildi (Sentry için yes, public için no)
- ✓ SSL/HTTPS aktif
- ✓ CSP header ayarlandı
- ✓ Security headers (X-Frame-Options, vs)
- ✓ Gzip/Brotli compression aktif
- ✓ CDN ile statik asset'ler
- ✓ Cache headers doğru (immutable for hashed files)
- ✓ Error tracking (Sentry) aktif
- ✓ Analytics aktif (gerekirse)
- ✓ SEO meta tags (og:image, twitter:card)
- ✓ Robots.txt + sitemap.xml
- ✓ 404 page tanımlandı
- ✓ Database backup stratejisi (backend)
- ✓ Rollback plan var

Modül 23 Özeti

Code'unu üretime taşımak için tüm bilgilere sahipsin. Build, container, CI/CD, deployment, monitoring — modern DevOps yaklaşımı.

Neler Öğrendin?

- ✓ Production build optimizasyonları
- ✓ Environment management
- ✓ Docker multi-stage build
- ✓ Nginx config
- ✓ GitHub Actions CI/CD
- ✓ Deployment platformları
- ✓ Monitoring & error tracking
- ✓ Production checklist

Sıradaki Modül

Modül 24 — Micro-Frontend Architecture. TSE'de kullandığın micro-frontend mimari derinlemesine — Module Federation, app shell, shared state.